
Universidad de Guayaquil
Facultad de Ciencias Matemáticas y Físicas
Carrera de Software

Bitácora técnica

Prototipo web de BI asistido por IA

Estudiantes	Emily Dayan Robles Alvarado y Lesly Samay Velásquez Anchundia
Tema	Construcción asistida de un datamart, KPIs, insights y pronóstico
Repositorio local	/home/ceo/Proyectos/BI
Versión de la bitácora	0.1.3
Fecha de apertura	24 de agosto de 2026

Índice

1. Propósito y reglas de mantenimiento	1
2. Resumen del alcance vigente	1
3. Registro cronológico	1
3.1. 24 de agosto de 2026 - Inicio del repositorio y ambiente	1
3.1.1. Decisiones técnicas	2
3.1.2. Archivos y componentes principales	2
3.1.3. Incidentes y observaciones	3
4. Matriz de verificación de la fase	4
5. Próximos pasos	4
6. Historial del documento	4

1. Propósito y reglas de mantenimiento

Este documento conserva la trazabilidad técnica del prototipo: decisiones, cambios, verificaciones, incidentes, evidencias y trabajo pendiente. Debe actualizarse al final de cada sesión significativa y antes de etiquetar una entrega.

Reglas:

- No eliminar incidentes ni resultados fallidos; agregar su resolución cuando exista.
- Registrar comandos y resultados relevantes sin copiar contraseñas, tokens o datos sensibles.
- Relacionar cada hito con un commit, pull request o etiqueta cuando GitHub esté disponible.
- Regenerar el PDF con `make logbook` y comprobar visualmente sus páginas.
- Mantener el archivo fuente `bitacora.tex` y el PDF generado bajo control de versiones.

2. Resumen del alcance vigente

El anteproyecto corregido define una prueba de concepto académica con una fuente activa SQL Server y AdventureWorks en modo de sólo lectura. El flujo futuro extraerá metadatos, solicitará propuestas estructuradas a un LLM, exigirá aprobación humana y validaciones determinísticas, ejecutará un ETL básico hacia PostgreSQL y mostrará cinco KPIs, tres visualizaciones, insights explicables y un pronóstico mensual mediante regresión lineal con MAPE y RMSE.

El anexo institucional exige módulos de seguridad, parámetros, negocio y reportes. Se acordó un RBAC mínimo con usuarios, roles, permisos y menús por rol. En la fase actual existe únicamente el límite arquitectónico; no hay autenticación ni lógica de negocio implementada.

3. Registro cronológico

3.1. 24 de agosto de 2026 - Inicio del repositorio y ambiente

Actividad	Estado	Resultado
Revisión del ante-proyecto	Completado	Se inspeccionaron las 11 páginas, incluyendo alcance, exclusiones, objetivos, metodología, cronograma y requisitos mínimos institucionales.
Arquitectura base	Completado	Se fijaron React con Vite y TypeScript, FastAPI, PostgreSQL interno/datamart y SQL Server AdventureWorks como fuente externa.
Docker-first	Completado	El host sólo requiere Docker Compose. Se definieron etapas de desarrollo, prueba y producción para frontend y backend.
Bases reproducibles	Completado	Se añadieron imágenes inicializadoras para PostgreSQL y SQL Server. AdventureWorks se restaura desde el respaldo oficial en un volumen vacío.

Actividad	Estado	Resultado
Seguridad	Completado	El paquete security y las reglas del RBAC quedaron documentados, sin modelos ni endpoints funcionales.
Observabilidad	Completado	Se añadieron endpoints <code>/health/live</code> y <code>/health/ready</code> , más una pantalla de estado en React.
DevOps	Completado	Se prepararon CI, CD, Dependabot, plantilla de pull request y publicación de imágenes en GHCR.
Guía de réplica	Completado	Se documentaron el entorno aislado desde cero y el consumo de imágenes publicadas.
GitHub remoto	Completado	Repositorio privado <code>LFXAS/prototipo-bi-ia</code> , rama <code>main</code> publicada y remoto <code>origin</code> configurado.
Verificación completa	Completado	El conjunto se reconstruyó desde volúmenes vacíos; los cuatro servicios quedaron saludables y las pruebas automatizadas finalizaron correctamente.

3.1.1. Decisiones técnicas

- El repositorio definitivo se ubica en `/home/ceo/Proyectos/BI`; no se codifican rutas absolutas en Compose, por lo que otras máquinas pueden clonarlo en cualquier ubicación.
- El Compose predeterminado crea red, bases y volúmenes aislados. Usa los puertos anfitrión 55432 y 51433 para no interferir con instancias existentes.
- Los volúmenes Docker no se exportan como imágenes. El estado inicial se reconstruye con respaldos públicos, scripts y futuras migraciones versionadas.
- `ApplicationIntent=ReadOnly` no es un control suficiente. El usuario `bi_reader` recibe lectura y denegaciones explícitas de escritura en SQL Server.
- CD publicará las imágenes propias en GitHub Container Registry con etiquetas de rama, versión y SHA.

3.1.2. Archivos y componentes principales

- `compose.yaml`: aplicación, PostgreSQL y SQL Server autocontenidos.
- `compose.release.yaml`: ejecución sin compilar, usando GHCR.
- `backend/`: API, configuración, sesión PostgreSQL, salud, Alembic y scaffolding modular.
- `frontend/`: interfaz React, proxy hacia la API, pruebas y Nginx de producción.
- `infra/`: inicialización de PostgreSQL, restauración de SQL Server y compilador LaTeX.
- `docs/replication-guide.md`: manual operativo paso a paso.
- `.github/workflows/`: integración y entrega continua.

3.1.3. Incidentes y observaciones

1. La carpeta generada inicialmente contenía un directorio `.git` vacío y no era todavía un repositorio válido. Se resolvió inicializando Git sobre `main`; el commit base es `ead11fb`.
2. El primer intento de generar `package-lock.json` falló porque npm trató de escribir en una caché de sólo lectura dentro del directorio personal. Se resolvió con la caché temporal `/tmp/bi-npm-cache`; el lockfile quedó generado sin vulnerabilidades reportadas.
3. Una verificación de Compose se lanzó desde la subcarpeta `frontend`; no encontró los archivos raíz. Se repitió desde la raíz y las configuraciones de desarrollo y publicación resultaron válidas.
4. La primera construcción del backend detectó formato pendiente y un import moderno requerido por Ruff. Se corrigieron ambos; Ruff, Mypy y dos pruebas Pytest finalizaron correctamente.
5. La primera prueba del frontend duplicó la inicialización de `jest-dom`. Se retiró la importación redundante; ESLint, Vitest y Vite finalizaron correctamente.
6. npm detectó vulnerabilidades en las versiones iniciales de Vite y Vitest. Se actualizaron a las versiones corregidas 7.3.6 y 3.2.7; el lockfile registra cero vulnerabilidades.
7. La primera salud del frontend consultó `localhost`, que BusyBox resolvió como IPv6 mientras Vite escuchaba en IPv4. Se cambió la sonda a `127.0.0.1` y el contenedor quedó saludable.
8. La primera restauración creó `bi_reader`, pero una denegación demasiado amplia de `CONTROL` también bloqueó el permiso subordinado de conexión. Se eliminó esa denegación, la sonda de salud pasó a probar el usuario real y se reconstruyó todo desde volúmenes vacíos.
9. La primera restauración de AdventureWorks requiere red y puede tardar varios minutos. Los reinicios posteriores reutilizan el volumen persistente.
10. La primera ejecución de Actions finalizó correctamente, pero advirtió que varias acciones todavía declaraban Node.js 20. Se actualizaron a las versiones oficiales vigentes: `checkout@v7`, `upload-artifact@v7` y acciones Docker de generaciones 4, 6 y 7.
11. La búsqueda autorizada de tres correos no devolvió cuentas públicas y la API de colaboradores exige nombres de usuario. Las invitaciones quedan pendientes hasta obtener los identificadores GitHub o iniciar sesión en la interfaz web.

4. Matriz de verificación de la fase

Control	Estado	Evidencia esperada
Compose autocontenido válido	Completado	<code>docker compose config -quiet</code>
Compose de imágenes válido	Completado	<code>docker compose -f compose.release.yaml config -quiet</code>
Backend calidad y pruebas	Completado	Ruff, Mypy y 2 pruebas Pytest dentro de Docker
Frontend calidad y pruebas	Completado	ESLint, 1 prueba Vitest y build de Vite dentro de Docker
PostgreSQL disponible	Completado	Esquemas <code>app</code> y <code>mart</code> ; endpoint <code>/health/ready</code> en <code>ok</code>
AdventureWorks restaurada	Completado	71 tablas; <code>bi_reader</code> : lectura 1 e inserción 0
Interfaz disponible	Completado	Frontend saludable en el puerto 5173 y API accesible mediante proxy
PDF legible	Completado	Todas las páginas renderizadas e inspeccionadas sin cortes ni solapamientos
Git local	Completado	Rama <code>main</code> ; commit base <code>ead11fb</code>
GitHub/GHCR	Completado	CI 32747013480 y CD 32747013085 exitosas; cinco imágenes privadas publicadas

5. Próximos pasos

1. Obtener los nombres de usuario GitHub de las colaboradoras y enviar las invitaciones con permiso de escritura.
2. Congelar una etiqueta de ambiente inicial y registrar los digest de las imágenes.
3. Iniciar la siguiente iteración con migraciones del RBAC y parámetros, sin adelantar módulos de negocio.

6. Historial del documento

Versión	Fecha	Cambio
0.1.0	24-08-2026	Apertura de la bitácora y registro de la fase de ambiente, Docker, Git y DevOps.
0.1.1	24-08-2026	Prueba integral desde volúmenes vacíos, corrección del usuario lector e inicialización del repositorio Git.
0.1.2	24-08-2026	Ejecución desde la ruta definitiva y validación de la sesión GitHub de LFXAS.
0.1.3	24-08-2026	Repositorio privado, primera CI/CD exitosa, imágenes GHCR y mantenimiento de Actions.